INFOM227 Program analysis for cybersecurity
Project 1 - Abstract interpretation (January)

Teaching assistant: Lucas Berg (lucas.berg@unamur.be)

2026 - 2027

1 Overview of the project

In this project, you will be asked to complete a study and implement a static program analysis tool based on an informal problem description. You can choose between 5 subjects all based on an extension of the Small programming language:

- An extension that **considers elements of concurrency**.
- An extension that **supports arrays**.
- An extension that **supports pointers**.
- An extension that **supports type annotations**.
- An extension that **supports exceptions**.

2 Delivrables

You are expected to deliver to WebCampus, in the space provided, an archive in standard format (.zip, .rar, .7z or other) containing:

- **a report in PDF**
- **the source code of your analysis tool**

3 Subject selection

An extension that considers elements of concurrency

This subject consists of extending the Small programming language so that it can take account of concurrency aspects. This means that portions of code can be executed in parallel. It is up to you to come up with a set of instructions to make this possible. After defining the new syntax and semantics, you will have to build an analyser that can find race conditions. The aim is to identify cases where the same resource (variable) is potentially required by two different threads, without being able to be certain of the order of execution of the two instructions using the resource. For example, a race condition can occur very easily when a read-modify-write operation is performed in a program using several threads. Here is an example of C code where the counter is not set to the proper value because of a read-modify-write race condition:

```c
#include <stdio.h>
#include <pthread.h>

int counter = 0;  // Shared resource

void *increment(void *arg) {
    for (int i = 0; i < 1000000; i++) {
        counter = counter + 1;  // Critical section: multiple threads can modify the resource
    }
    return NULL;
}

int main(void) {
    pthread_t thread1, thread2;

    // Create two threads
    pthread_create(&thread1, NULL, increment, NULL);
    pthread_create(&thread2, NULL, increment, NULL);

    // Wait for both threads to finish
    pthread_join(thread1, NULL);
    pthread_join(thread2, NULL);

    // Final value of counter
    printf("Final counter value: %d\n", counter);  // Expected: 2000000 but is usually less due to a race condition

    return 0;
}
```

Listing 1: Example of race condition in C

An extension that supports arrays

This subject consists of extending the Small programming language so that it can support arrays. It is up to you to come up with a set of instructions to make this possible. After defining the syntax and semantics, you will have to build an analyser that can detect errors caused by the use of arrays using an **interprocedural analysis**. Several problems can occur when using arrays, such as buffer overflows, crashes and many others, and your goal is to detect some of these errors. Here is an example of C code that does not validate the index used by the array and could cause some problems:

```c
#include <stdio.h>

int main(void) {
    int index;
    int array[] = {1, 2, 3};

    scanf("%d", &index);

    array[index] = 0;

    return 0;
}
```

Listing 2: Example of invalid use of array in C

An extension that supports pointers

This subject consists of extending the Small programming language so that it can support pointers. It is up to you to come up with a set of instructions to make this possible. After defining the syntax and semantics, you will have to build an analyser that can detect errors caused by the use of pointers, using an **interprocedural analysis if possible**. Several problems can occur when using pointers,

such as null pointer dereference, dangling pointers and many others, and your goal is to detect some of these errors. Here is an example of C code that tries to use a dangling pointer:

```c
#include <stdio.h>

int main(void) {
    int *variable;

    printf("variable: %d\n", *variable);

    return 0;
}
```

Listing 3: Example of dangling pointer in C

An extension that supports type annotations

This subject consists of extending the Small programming language so that it can support type annotations. It is up to you to come up with a set of instructions to make this possible. After defining the syntax and semantics, you will have to build an analyser that can statically detect errors caused by the incorrect use of type operations, such as adding an `int` to a `bool`. **Type annotations are optional, so your analysis must be capable of performing type inference.** It is a technique commonly used in dynamic programming languages such as Python or JavaScript. Here is an example of Python code that try to call a function with an argument that has the wrong type:

```python
def double(x: int) -> int:
    return x * 2

value = double("str")  # "double" only accepts an "int" as an argument, not a "str"
```

Listing 4: Example of invalid function call

An extension that supports exceptions

This subject consists of extending the Small programming language so that it can support exceptions. It is up to you to come up with a set of instructions to make this possible. After defining the syntax and semantics, you will have to build an analyser that can detect some errors caused by uncaught exceptions using an **interprocedural analysis**. Here is an example of Python code that does not catch a `ZeroDivisionError` exception :

```python
def mean(ls: list[float]) -> float:
    result = 0.0
    for element in ls:
        result += element
    return result / len(ls)

value = mean([])  # "ls" is empty, so "result / len(ls)" raises a ZeroDivisionError
```

Listing 5: Example of invalid function call

4 Ambiguities

The 5 subjects described above clearly contain ambiguities: we have not said, for example, based on which criteria a race condition is considered problematic or which errors to focus on. One of the goals of the project is for you to research your subject and make decisions which will inevitably distinguish your solution from those proposed by your colleagues.

5 Instructions

Implementation

Your analyser should be based on the ExampleAnalyser from the practical sessions, written in Scala. However, if you really want to start from scratch in another language, you can send an email to **lucas.berg@unamur.be** to justify your request.

Your analyser will have to take the path of a file containing the code as an argument so that the file can be analysed. Then, it should run automatically and display the error and warning messages. For the rest, this work is exploratory. You are free to clarify any concepts left unclear in the project. It is normal for you to have to explore the subject by doing some research (online, etc.) to better understand the type of analysis you are going to implement. **You should cite your external sources in the report**.

Report

The report should explain clearly, formally, precisely and unambiguously the following points:

1. An informal description of your analyser: What are its objectives, capabilities, limitations ?
2. The changes in syntax and semantics.
3. The abstract domain and the lattice on which you rely.
4. The abstraction function and the concretisation function.
5. One flow function that you consider interesting and the proof that it is sound based on the semantics.
6. An explanation of how your analysis works: Which property does it compute, how does it process the program (forward or backward, why?) and in which cases errors are reported ?
7. An example input program for which you will explain, step by step, the calculations performed by your analysis.
8. The procedure for installing and executing your program.
9. Any other information that could be useful to provide.

6 Evaluation

This project and its oral defense is worth **50%** of the course mark.

The main elements of the evaluation are:

- Your ability to discuss in general the concepts and theories seen in the course, and in full technical detail those relevant to your project
- The accuracy and appropriateness of the program provided
- The quality of the content and presentation of the report
- The consistency of the specification and implementation choices made

You are responsible for the code that you submit. It is therefore important that you can explain in full technical detail what the code does, be able to justify each and every design or implementation decision made, and link all of this with the formal concepts and theory studied in the course. If you have used AI, you must demonstrate that you took appropriate steps to take ownership of and understand your code.

7 Deadline

The project must be delivered by **03 January 2027 at 23:59**.

8 Any questions?

Prefer the Q&A forum on WebCampus or send an email or a Teams message to **lucas.berg@unamur.be**.

Good luck!